ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Đoàn Văn Giáp

NGHIÊN CỨU VỀ ĐỘNG LỰC NỘI TẠI TRONG HỌC TĂNG CƯỜNG

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Đoàn Văn Giáp

RESEARCH ON INTRINSIC MOTIVATION FOR REINFORCEMENT LEARNING

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: TS. Nguyễn Thị Thủy

HÀ NỘI - 2026

VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Doan Van Giap

RESEARCH ON INTRINSIC MOTIVATION FOR REINFORCEMENT LEARNING

BACHELOR'S THESIS

Major: Information Technology

Supervisor: Dr. Nguyen Thi Thuy

HANOI - 2026

Acknowledgement

I would like to express my sincere gratitude to my supervisor and co-supervisor for their guidance, encouragement, and valuable feedback throughout the completion of this thesis. I also thank my lecturers, classmates, and family members for their continuous support during my study and research process.

Declaration of Authorship

I hereby declare that this thesis is my own work and has not been submitted previously for a degree or diploma at this or any other higher education institution. To the best of my knowledge and belief, the thesis contains no material previously published or written by another person except where proper acknowledgment and citation are given.

Hanoi, ...

Student

Doan Van Giap

Tóm tắt

Khóa luận *Research on Intrinsic Motivation for Reinforcement Learning* nghiên cứu bài toán học tăng cường trong môi trường phần thưởng thưa, nơi tác tử rất khó khám phá do tín hiệu phản hồi từ môi trường xuất hiện không thường xuyên. Nhiều phương pháp hiện nay kết hợp phần thưởng ngoại sinh với phần thưởng nội tại dựa trên tò mò hoặc độ mới, nhưng thường phải dùng một hệ số cố định để cân bằng hai nguồn phần thưởng này. Hệ số cố định dễ nhạy với từng bài toán và không phản ánh được việc ở những trạng thái khác nhau thì nhu cầu khám phá cũng khác nhau.

Khóa luận đề xuất **Critic-Adaptive Reward Exploration (CARE)**, một cơ chế điều chỉnh phần thưởng nội tại theo từng trạng thái thông qua một Beta Network gọn nhẹ. Mạng này được huấn luyện theo mục tiêu learning progress: Beta Network bám theo độ lớn của lỗi temporal-difference (TD-error) của value function ngoại sinh, nhờ đó phần thưởng nội tại được khuếch đại tại những trạng thái mà tác tử đang học và bị giảm tại những trạng thái đã ổn định. Phương pháp được áp dụng cho ba họ phần thưởng nội tại tiêu biểu là count-based, Intrinsic Curiosity Module (ICM), và Rewarding Impact-Driven Exploration (RIDE), tất cả đều kết hợp với PPO làm thuật toán tối ưu chính.

Kết quả thực nghiệm trên sáu môi trường MiniGrid (DoorKey-8x8, Empty-16x16, KeyCorridor-S3R3, LavaCrossing-S9N3, RedBlueDoors-8x8, UnlockPickup) cho thấy CARE cải thiện hiệu quả mẫu và giảm độ dao động so với PPO thuần và so với mọi cấu hình hệ số cố định $\beta \in \{0.001, 0.005, 0.05, 0.1\}$ của từng module. Phương pháp đặc biệt hiệu quả trong các môi trường có phần thưởng thưa nhưng vẫn còn tín hiệu ngoại sinh mang tính định hướng. Trong những bài toán quá thưa, CARE suy giảm một cách ổn định về gần hành vi của hệ số cố định thay vì gây mất ổn định cho quá trình học.

Từ khóa: học tăng cường, phần thưởng thưa, phần thưởng nội tại, khám phá dựa trên tò mò, điều chỉnh phần thưởng thích ứng, ICM, RIDE

Abstract

Sparse-reward reinforcement learning remains difficult because agents receive limited feedback about whether their behavior is useful. A common solution is to augment the environment reward with intrinsic motivation signals such as novelty, curiosity, or impact. However, count-based bonuses, the Intrinsic Curiosity Module (ICM), and Rewarding Impact-Driven Exploration (RIDE) are all commonly combined with extrinsic rewards through a fixed coefficient. This design makes performance highly sensitive to manual tuning and obscures a broader empirical question: how does an adaptive scaling mechanism affect different intrinsic reward modules?

This thesis proposes **Critic-Adaptive Reward Exploration (CARE)**, a module-agnostic adaptive wrapper for intrinsic reward modules. CARE learns a state-dependent scaling factor through a lightweight Beta Network and uses a learning-progress objective: the Beta Network is trained to track the magnitude of the temporal-difference error of the extrinsic value function, so that intrinsic reward is amplified at states where the agent is still learning and attenuated at states it has already mastered. Under this framing, CARE is not treated only as a standalone method paired with one curiosity bonus, but as a mechanism for studying and improving how representative intrinsic reward modules are used during training.

The thesis therefore analyzes CARE with respect to three representative intrinsic reward families: count-based exploration, ICM, and RIDE. The quantitative experiments instantiate the framework with Proximal Policy Optimization on six sparse-reward MiniGrid environments (DoorKey-8x8, Empty-16x16, KeyCorridor-S3R3, LavaCrossing-S9N3, RedBlueDoors-8x8, and UnlockPickup). For each intrinsic module, the adaptive CARE variant is compared against a sweep of fixed scalar coefficients $\beta \in \{0.001, 0.005, 0.05, 0.1\}$, together with a no-intrinsic PPO control. The results show that learned state-dependent scaling improves sample efficiency and reduces variance relative to PPO without intrinsic rewards and to the best fixed-β configuration of each module. Overall, the study suggests that CARE is best understood as a practical adaptive layer whose benefits depend on the structure and informativeness of the underlying intrinsic reward module.

Keywords: reinforcement learning, sparse rewards, intrinsic motivation, adaptive reward scaling, count-based exploration, ICM, RIDE

Contents

List of Figures

List of Tables

List of Acronyms

Acronym	Full Form	Description
CARE	Critic-Adaptive Reward Exploration	Proposed adaptive intrinsic reward scaling framework
RL	Reinforcement Learning	Learning paradigm based on sequential interaction with an environment
PPO	Proximal Policy Optimization	On-policy policy-gradient algorithm used as the training backbone
ICM	Intrinsic Curiosity Module	Intrinsic reward model based on forward prediction error
RND	Random Network Distillation	Curiosity-based method using predictor error against a random target network
RE3	Random Encoders for Efficient Exploration	Curiosity-based method using k-NN state entropy in a fixed random encoder space
RIDE	Rewarding Impact-Driven Exploration	Intrinsic reward method based on state-change impact
GAE	Generalized Advantage Estimation	Technique for reducing variance in policy-gradient advantage estimates
MDP	Markov Decision Process	Standard formal model for reinforcement learning environments

List of Notations

Markov Decision Process and Policy

Symbol	Meaning
$\mathcal{S}$, $\mathcal{A}$	State space and action space
$P(s' \mid s, a)$	State transition kernel
ρ_0	Initial state distribution
$\gamma \in [0, 1)$	Discount factor
$\pi_\theta(a \mid s)$	Stochastic policy with parameters θ
$V_\mu(s)$	State-value function with parameters μ
$\hat{A}_t$	Generalized advantage estimate at time t
$\rho_t(\theta)$	PPO importance ratio $\pi_\theta(a_t \mid s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t \mid s_t)$

Reward Signals

Symbol	Meaning
R_t^E	Extrinsic reward from the environment at time t
R_t^I	Generic intrinsic reward at time t (any module)
$R_t^{I,m}$	Intrinsic reward at time t produced by module m
I_t	Raw intrinsic reward (forward prediction error in the ICM case)
I_t^+, $I_t^{+,m}$	Standardized and rectified intrinsic reward (generic / module-specific)
$\tilde{I}_t^m$	Scaled intrinsic reward $\beta_\psi(s_t)\, I_t^{+,m}$
$\bar{r}_t$	Shaped reward used by PPO
δ_t	One-step extrinsic temporal-difference error of the critic at time t
$\tau(s_t)$	Per-state target for β_ψ derived from $\lvert\delta_t\rvert$

ICM Components

Symbol	Meaning
$\phi(s_t)$	ICM state encoder
f_η^F	ICM forward dynamics model with parameters η
f_η^I	ICM inverse dynamics model with parameters η
α_F, α_I	Forward and inverse loss weights in L_{ICM}

CARE Components

Symbol	Meaning
$\beta_\psi(s)$	State-dependent scaling function with parameters ψ
$[\beta_{\min}, \beta_{\max}]$	Output range of the Beta Network
β_0	Reference value for log-space regularization
γ_p	Progress-loss weight in the Beta loss
λ_{reg}	Regularization weight in the Beta loss

Loss Functions

Symbol	Meaning
$L^{\text{PPO}}(\theta)$	PPO clipped surrogate actor loss
$L^V(\mu)$	Critic mean-squared-error loss
$L_{\text{ICM}}(\eta)$	ICM combined forward + inverse loss
$L_{\text{progress}}(\psi)$	Learning-progress (target-tracking) component of the Beta loss
$L_{\text{reg}}(\psi)$	Log-space regularization component of the Beta loss
$L_\beta(\psi)$	Total Beta-Network loss $\gamma_p\, L_{\text{progress}} + \lambda_{\text{reg}}\, L_{\text{reg}}$

Training Schedule

Symbol	**Meaning**
T	Rollout buffer length
U	Total number of PPO updates
K	Number of PPO gradient epochs per update
ξ_t	Single transition tuple stored in the rollout buffer
d_t	Episode termination flag at time t (1 if terminal, 0 otherwise)

Chapter 1

Introduction

> *"The important thing is not to stop questioning. Curiosity has its own reason for existing."*
>
> — Albert Einstein

Curiosity is widely viewed as a driving force behind learning, both in humans and in artificial systems. In reinforcement learning (RL), this idea has been formalized as intrinsic motivation: an internally generated signal that encourages an agent to seek out novel or informative experiences when external feedback is scarce. Yet the question of *how strongly* an agent should listen to its own curiosity remains largely unresolved. Most existing methods rely on a single, manually chosen coefficient that controls the influence of intrinsic rewards uniformly across all states. This thesis takes the position that the importance of curiosity is itself state-dependent and proposes a learned scaling mechanism that adapts the influence of intrinsic motivation to the current situation of the agent.

1.1 Background and Research Motivation

Reinforcement learning has emerged as a central paradigm for training agents to make sequential decisions in complex environments, with influential successes in Atari games, Go, and robotic control [1–3]. Many of these results, however, depend on dense reward signals that provide frequent feedback about which behaviors are desirable. In contrast, a wide range of practical environments are sparse-reward: the agent receives meaningful feedback only after a long sequence of actions, which makes exploration substantially harder [4].

To address this difficulty, a large body of research augments the extrinsic reward supplied by the environment with an additional intrinsic reward that encourages exploration [5, 6]. Count-based methods reward the agent for visiting rarely seen states [7–9]. Curiosity-driven methods derive intrinsic rewards from prediction errors, representation discrepancies, or state-entropy estimates, as in the Intrinsic

Curiosity Module (ICM) [10], Random Network Distillation (RND) [11], Rewarding Impact-Driven Exploration (RIDE) [12], and Random Encoders for Efficient Exploration (RE3) [13]. These approaches improve exploration in many tasks, but they all share a key design choice: the strength with which intrinsic rewards influence learning relative to extrinsic rewards.

In standard practice, this strength is controlled by a fixed scalar coefficient that must be tuned manually for each environment. The optimal value of this coefficient often varies across tasks and even across different stages of training [14, 15]. More fundamentally, a single global coefficient cannot distinguish between states where exploration remains useful and states where exploration has become distracting or harmful. The same limitation appears across multiple intrinsic reward families, from count-based bonuses to ICM and RIDE. This shared weakness motivates a broader research perspective: rather than studying yet another curiosity module, this thesis investigates an adaptive mechanism whose effect can be analyzed across representative intrinsic reward modules.

1.2 Research Problem

This thesis investigates the following problem: how does adaptive, state-dependent intrinsic reward scaling affect different intrinsic reward modules, and can such scaling promote useful exploration in sparse-reward reinforcement learning without destabilizing policy optimization?

More specifically, let the agent receive an extrinsic reward R_t^E from the environment and an intrinsic reward $R_t^{I,m}$ produced by intrinsic reward module m, where m may denote a count-based bonus, ICM, or RIDE. Standard methods optimize a shaped reward of the form

$$\bar{r}_t = R_t^E + \beta R_t^{I,m}, \tag{1.1}$$

where β is a fixed hyperparameter. The limitation of this formulation is that the same intrinsic scaling is applied uniformly to all states, even though the value of exploration may differ substantially across the state space. The central research question of this thesis is therefore:

> *Can a state-dependent scaling mechanism, learned online from the agent's own experience, improve how representative intrinsic reward modules such*

as count-based bonuses, ICM, and RIDE contribute to learning in sparse-reward environments?

1.3 Research Objectives and Scope

The objective of this thesis is to develop and empirically evaluate **Critic-Adaptive Reward Exploration (CARE)**, a state-dependent scaling mechanism for intrinsic rewards, and to study its effect on representative intrinsic reward modules in sparse-reward reinforcement learning. The specific objectives are as follows:

- analyze why fixed intrinsic reward coefficients are a shared limitation across representative intrinsic reward modules;
- review count-based exploration, ICM, RIDE, and prior adaptive reward weighting approaches as the foundation for state-dependent adaptation;
- formulate CARE as a lightweight, module-agnostic scaling layer that learns a state-dependent coefficient $\beta(s_t)$ via a learning-progress objective driven by the temporal-difference error of the value function;
- describe the concrete implementation of CARE with Proximal Policy Optimization (PPO) and three representative intrinsic reward modules (count-based bonuses, ICM, and RIDE), each compared against a sweep of fixed scalar coefficients;
- evaluate the empirical effect of adaptive scaling in sparse-reward MiniGrid environments and discuss what the results imply for different intrinsic reward modules.

The scope of this thesis is limited to episodic reinforcement learning tasks with sparse rewards. Conceptually and empirically, the study is framed around three representative intrinsic reward families: count-based bonuses, ICM, and RIDE. The quantitative experiments use PPO as the base policy optimization algorithm and instantiate the framework with all three intrinsic reward modules; for each module, the adaptive CARE variant is compared against a sweep of fixed scalar coefficients $\beta \in \{0.001, 0.005, 0.05, 0.1\}$ together with a no-intrinsic PPO control. The experiments focus on benchmark MiniGrid environments [16] rather than real-world

robotics or continuous-control tasks. The thesis emphasizes empirical performance and methodological clarity rather than a full theoretical proof of convergence or optimality.

1.4 Research Methodology

The work follows a research-oriented methodology with four stages. First, the thesis surveys foundational RL literature and representative intrinsic reward modules to identify the common limitation of fixed intrinsic weighting. Second, it formulates CARE as an adaptive scaling mechanism that predicts a positive coefficient $\beta(s_t)$ from the current state and can be applied to module-specific intrinsic rewards. Third, the method is instantiated within a PPO training loop with three representative intrinsic reward modules (count-based bonuses, ICM, and RIDE). Finally, the approach is validated experimentally on multiple sparse-reward tasks in MiniGrid by comparing it against PPO without intrinsic rewards and against each module with a swept set of fixed intrinsic coefficients.

The evaluation criteria emphasize sample efficiency, learning stability, robustness across environments, and the qualitative behavior of the learned scaling function. This combination of theoretical grounding, algorithmic design, and empirical validation supports an experimental framing centered on the effect of CARE on intrinsic reward modules rather than on the construction of a new intrinsic reward generator.

1.5 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 reviews reinforcement learning in sparse-reward environments, representative intrinsic reward modules, and prior attempts to adapt the weighting of intrinsic rewards; it also identifies the research gap that motivates the proposed method. Chapter 3 presents the proposed methodology, including the formal problem statement, the module-agnostic CARE formulation, the learning-progress training objective, the three intrinsic module instantiations (count-based, ICM, RIDE), and the complete training algorithm; it also states five formal results (Propositions, a Lemma, and a Corollary) that characterize CARE's properties. Chapter 4 describes the experimental setup,

including the MiniGrid benchmark environments, the implementation details, the evaluation metrics, and the analysis of results from the perspective of CARE's effect on intrinsic reward modules. The conclusion summarizes the main findings, discusses limitations of the proposed approach, and outlines directions for future research. Two appendices complete the thesis: Appendix A contains the full proofs of the formal results introduced in Chapter 3, and Appendix B records the network architectures, hyperparameters, and training schedule used in the experiments.

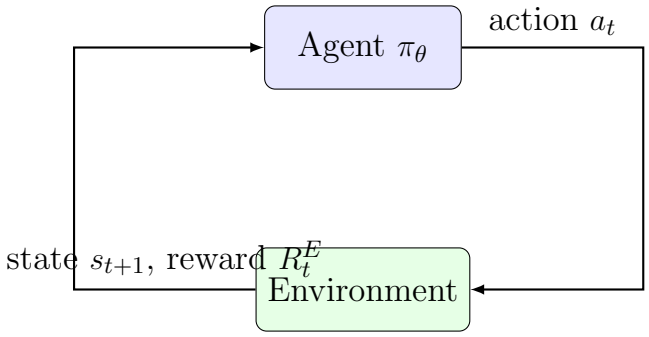
Chapter 2

Literature Review

This chapter summarizes the published studies most relevant to this thesis. It first formalizes reinforcement learning in sparse-reward environments and discusses why standard exploration is insufficient in such settings. It then reviews representative intrinsic motivation methods, including count-based and curiosity-driven exploration, and surveys prior attempts to adapt the weighting of intrinsic rewards. The chapter concludes by identifying the specific gap that motivates the proposed methodology in Chapter 3.

2.1 Reinforcement Learning in Sparse-Reward Environments

An episodic reinforcement learning (RL) problem is commonly modeled as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s_{t+1} \mid s_t, a_t)$ denotes the transition dynamics, R is the reward function, $\gamma \in [0, 1)$ is the discount factor, and ρ_0 is the initial state distribution [3]. At each time step t, the agent observes a state $s_t \in \mathcal{S}$, samples an action $a_t \sim \pi_\theta(\cdot \mid s_t)$ from a parameterized policy, receives an extrinsic reward R_t^E, and transitions to a new state s_{t+1}. This standard interaction loop is depicted in Figure 2.1.



Hình 2.1: Standard agent–environment interaction loop in reinforcement learning. The agent observes a state, executes an action sampled from its policy, and receives a new state along with an extrinsic reward from the environment.

The objective of policy optimization is to learn parameters θ that maximize the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim (\pi_\theta, P)} \left[\sum_{t=0}^{T-1} \gamma^t R_t^E \right], \tag{2.1}$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ denotes a trajectory induced by the policy and the transition dynamics.

Although Equation (2.1) is a standard formulation, learning becomes difficult when extrinsic rewards are sparse. In sparse-reward tasks, the agent may take a long sequence of actions before receiving any feedback that distinguishes promising trajectories from uninformative ones. Classical exploration strategies such as ϵ-greedy or stochastic policies inject randomness into action selection, but they do not actively guide the agent toward informative experiences when the state space is large or partially observable [4]. As a result, an agent relying solely on extrinsic rewards in such environments often fails to discover the goal-reaching behaviors needed for learning to begin.

2.2 Intrinsic Motivation for Exploration

Intrinsic motivation addresses sparse rewards by supplementing the environment's reward with an additional exploration bonus that is internally generated by the agent itself [5, 6]. Under this paradigm, the agent receives both an extrinsic reward R_t^E and an intrinsic reward R_t^I, and policy optimization is performed on a shaped objective:

$$\bar{r}_t = R_t^E + \beta R_t^I, \tag{2.2}$$

where $\beta \geq 0$ controls the relative importance of intrinsic motivation. The intrinsic signal R_t^I is designed to be larger for states or transitions that the agent finds novel, surprising, or otherwise informative. A recent survey by Aubret *et al.* [17] unifies many of these methods through an information-theoretic perspective, showing that seemingly different intrinsic reward generators can be understood as different practical instantiations of similar underlying principles.

2.2.1 Count-Based Exploration

Count-based methods assign higher intrinsic rewards to states that have been visited less frequently. In tabular settings, the intrinsic reward at state s is often inversely proportional to the square root of its visit count $N(s)$, capturing the intuition that under-explored states are more valuable to revisit. Bellemare *et al.* [7] extend this principle to high-dimensional observations through pseudo-counts derived from density models, while Tang *et al.* [8] use random projection hashing to obtain approximate counts in continuous state spaces. Ostrovski *et al.* [9] further refine the approach with neural density models, demonstrating strong exploration on hard Atari games such as *Montezuma's Revenge*. Count-based methods are conceptually simple and often effective, but the quality of the intrinsic signal is bounded by the ability of the underlying density estimator to reliably quantify novelty in complex observation spaces.

2.2.2 Curiosity-Driven Exploration

Curiosity-driven methods define intrinsic rewards through some form of prediction error. The Intrinsic Curiosity Module (ICM) [10] learns a compact latent representation of the state and trains a forward model that predicts the next latent feature from the current feature and action. The prediction error of this forward model is then used as an intrinsic bonus, encouraging the agent to seek transitions whose dynamics it cannot yet predict accurately. Random Network Distillation (RND) [11] replaces the forward dynamics model with the discrepancy between a fixed random target network and a learned predictor, providing a measure of state novelty that does not depend on action prediction. Rewarding Impact-Driven Exploration (RIDE) [12] computes intrinsic rewards from the magnitude of representation changes that are causally induced by the agent's actions, which makes it particularly suited to procedurally generated environments where stochastic background changes might otherwise dominate prediction-based bonuses. More recently, Random Encoders for Efficient Exploration (RE3) [13] replaces learned predictors entirely: it estimates state entropy by computing k-nearest neighbor distances in the representation space of a fixed, randomly initialized encoder. Because the encoder is never trained, RE3 has no learnable parameters in its intrinsic reward branch and is therefore particularly compute-efficient.

A common advantage of curiosity-driven methods is that intrinsic rewards tend to decrease naturally as the agent becomes familiar with a region of the state space, reducing the risk of repeatedly exploring the same area. However, this self-correcting property only operates at the level of the raw intrinsic signal; the relative weight of that signal in the shaped reward of Equation (2.2) is still controlled by an external coefficient β that does not adapt to context.

2.3 Adaptive Weighting of Intrinsic Rewards

Despite the diversity of intrinsic motivation modules, most of them depend on a manually selected coefficient β in the shaped reward of Equation (2.2) [10, 11]. This fixed coefficient creates a structural limitation. If β is too small, the intrinsic signal becomes negligible and exploration remains weak. If β is too large, curiosity may dominate the training objective and distract the agent from the underlying task. Because the optimal trade-off varies across environments and even across different stages of the same training run, fixed weighting lacks flexibility. Several lines of research have attempted to overcome this limitation, with adaptation operating at progressively finer granularity.

2.3.1 Global-Level Adaptation

A first family of approaches treats the trade-off between extrinsic and intrinsic rewards as a global optimization problem. Extrinsic-Intrinsic Policy Optimization (EIPO) [14] casts intrinsic reward weighting as a constrained optimization in which the influence of intrinsic motivation is increased only when the extrinsic objective allows for it, preventing intrinsic rewards from harming task performance. Automatic Intrinsic Reward Shaping (AIRS) [15] frames the selection among different intrinsic reward functions as a bandit-style decision process, automatically choosing which exploration signal is most beneficial at each phase of training. Both methods improve over hand-tuned coefficients, but their adaptation is global: a single weight or a single intrinsic reward type is selected for the entire policy at any given time, regardless of the specific state being visited.

2.3.2 Population-Level Adaptation

A second family of approaches coordinates a population of policies with different exploration–exploitation trade-offs. Never Give Up (NGU) [18] maintains multiple value functions associated with different intrinsic reward weights, allowing the agent to share a single neural network across distinct exploratory regimes. Agent57 [19] extends this idea by additionally adapting the discount factor and weighting through a meta-controller that selects among the population members based on their recent performance. These methods substantially improve robustness across diverse Atari benchmarks, but their adaptation occurs at the level of an entire policy, not at the level of individual states. The shaping of intrinsic rewards within any single policy of the population is still governed by a fixed coefficient.

2.3.3 Toward State-Level Adaptation

A third, more direct line of work attempts to learn the intrinsic reward signal itself. Zheng *et al.* [20] propose Learning Intrinsic Rewards for Policy Gradient methods (LIRPG), in which a parameterized intrinsic reward function is trained via meta-gradients so that, after a policy update using the shaped reward, the resulting policy maximizes the extrinsic objective. This formulation is conceptually appealing because the intrinsic reward is directly aligned with future extrinsic performance. In practice, however, meta-gradient training requires differentiating through one or more policy updates, which introduces substantial computational overhead and can be unstable in deep RL settings.

The conceptual gap that emerges from this body of work is clear. Existing adaptive methods either operate at a coarse granularity (global or population-level) or rely on computationally expensive higher-order optimization (LIRPG-style meta-gradients). What is missing is a lightweight mechanism that adapts the influence of intrinsic rewards at the state level, learned via a first-order objective, and that can be plugged on top of different intrinsic reward generators without modifying their internal mechanics.

2.4 Summary and Research Gap

This chapter has reviewed the foundations on which the proposed work is built. Reinforcement learning in sparse-reward environments is challenging because conventional exploration strategies provide only weak guidance. Intrinsic motivation methods such as count-based bonuses, ICM, RND, RIDE, and RE3 alleviate this difficulty by injecting an internally generated exploration signal, but the strength of that signal is typically governed by a fixed scalar coefficient. Prior work on adaptive weighting addresses this limitation only partially: global and population-level adaptation modulate intrinsic motivation at coarse granularity, while meta-gradient methods such as LIRPG operate at the right level but at a high computational cost.

Two related research gaps emerge from this analysis. First, there is no lightweight, state-dependent mechanism that modulates intrinsic rewards according to how well they correlate with future extrinsic outcomes, despite the fact that two states with similar novelty scores can contribute very differently to downstream task success. Second, although count-based bonuses, ICM, and RIDE share the same fixed-coefficient pattern, the literature does not provide a unified experimental framing that asks whether a single adaptive mechanism can improve them in a consistent way.

The methodology presented in Chapter 3 is designed to address both gaps. It introduces a state-dependent scaling function that is trained with a first-order correlation-based objective and that operates as a module-agnostic adaptive layer above representative intrinsic reward generators. The remainder of the thesis develops this approach in detail and evaluates its empirical effect on intrinsic reward modules in sparse-reward MiniGrid benchmarks.

Chapter 3

Methodology

This chapter presents the proposed methodology in formal terms. It first defines the underlying intrinsic-augmented decision process and the overall system architecture. It then specifies the intrinsic reward generator used in the implemented system, formulates the Critic-Adaptive Reward Exploration (CARE) framework with its Beta Network and learning-progress objective, and concludes with the complete training algorithm. Five formal results (Propositions 3.1–3.3, Lemma 3.1, Corollary 3.1) are stated in this chapter and proved in Appendix A.

3.1 System Architecture

The proposed framework is built on top of an intrinsic-augmented Markov decision process (MDP) and introduces a learned, state-dependent scaling function for intrinsic rewards. The two ingredients are made explicit in the following definitions.

Definition 3.1 (Intrinsic-Augmented MDP)**.** An *intrinsic-augmented MDP* is a tuple $(\mathcal{S}, \mathcal{A}, P, R^E, R^I, \gamma, \rho_0)$ in which $(\mathcal{S}, \mathcal{A}, P, R^E, \gamma, \rho_0)$ defines a standard MDP with extrinsic reward function $R^E : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, and $R^I : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}_{\geq 0}$ is an additional, internally generated intrinsic reward function. The agent observes only $(s_t, a_t, R^E_t, s_{t+1})$ from the environment; the intrinsic reward R^I_t is computed by the agent itself.

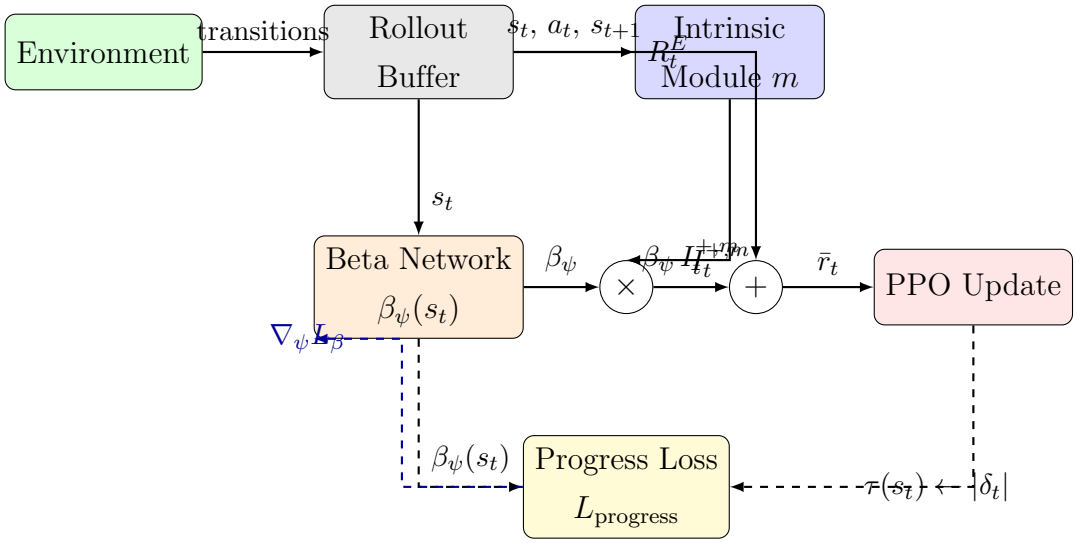
Definition 3.2 (State-Dependent Reward Shaping)**.** A *state-dependent reward shaping function* is a measurable map $\beta : \mathcal{S} \to \mathbb{R}_+$. Given an intrinsic-augmented MDP, the corresponding shaped reward at time step t is

$$\bar{r}_t = R^E_t + \beta(s_t)\, R^I_t. \tag{3.1}$$

The standard fixed-coefficient formulation corresponds to the special case $\beta(s) \equiv \beta_0$ for some constant $\beta_0 > 0$.

The proposed system instantiates this formulation with a parameterized β_ψ together with one of three intrinsic reward generators — count-based exploration,

the Intrinsic Curiosity Module, or Rewarding Impact-Driven Exploration — all of which are evaluated in Chapter 4. The full pipeline is shown in Figure 3.1.



Hình 3.1: CARE system architecture. The intrinsic module block stands for any of the three implementations evaluated in this thesis (count-based, ICM, or RIDE); the rest of the pipeline is identical across modules. Solid arrows trace the forward computation that produces the shaped reward $\bar{r}_t$ used by PPO. Dashed arrows indicate the auxiliary signals used to update the Beta Network β_ψ via the learning-progress objective L_β, with the $\tau(s_t)$ target derived from the magnitude of the critic's TD error δ_t.

The shaped reward used throughout this thesis follows directly from Definition 3.2. For the chosen intrinsic reward module m with rectified intrinsic signal $I_t^{+,m}$ (Section 3.2.4), the combined reward optimized by PPO is

$$\bar{r}_t \;=\; R_t^E \;+\; \beta_\psi(s_t)\, I_t^{+,m}. \tag{3.2}$$

Conceptually, $\beta_\psi(s_t)$ acts as a state-conditional volume control on curiosity: it amplifies intrinsic reward in states where exploration is empirically associated with future extrinsic success and attenuates it elsewhere. The remainder of this chapter formalizes this intuition.

3.2 Intrinsic Reward Generation

The proposed framework supports three intrinsic reward modules: count-based exploration [7, 8], the Intrinsic Curiosity Module (ICM) [10], and Rewarding Impact-Driven Exploration (RIDE) [12]. All three differ only in how the raw intrinsic signal

I_t is computed at each transition; the rest of the pipeline (standardization, Beta Network, and learning-progress objective) is identical. The empirical evaluation in Chapter 4 reports on all three modules. The remainder of this section describes each module with its raw-reward formulation, followed by the shared standardization step that produces the rectified signal I_t^+ fed to the Beta Network.

3.2.1 ICM Instantiation

Forward and Inverse Dynamics Models. ICM first encodes each state into a latent representation $\phi(s_t) \in \mathbb{R}^d$. It then trains two auxiliary models: a forward dynamics model f_η^F that predicts the next latent feature from the current feature and action, and an inverse dynamics model f_η^I that predicts the action from two consecutive latent states. The combined ICM loss is

$$L_{\text{ICM}}(\eta) \;=\; \alpha_F \,\mathbb{E}\Big[\tfrac{1}{2}\big\|f_\eta^F(\phi_t, a_t) - \phi_{t+1}\big\|_2^2\Big] \;+\; \alpha_I \,\mathbb{E}[-\log p_\eta(a_t \mid \phi_t, \phi_{t+1})]\,, \tag{3.3}$$

with weighting coefficients $\alpha_F, \alpha_I > 0$.

Raw Intrinsic Reward. The raw intrinsic reward at time step t is the forward prediction error

$$I_t \;=\; \tfrac{1}{2}\big\|f_\eta^F(\phi_t, a_t) - \phi_{t+1}\big\|_2^2. \tag{3.4}$$

3.2.2 Count-Based Exploration

In the count-based instantiation, the raw intrinsic reward at state s_t is a function of the empirical visit count of that state. Following [8], we use a random-projection hash $\phi_h : \mathcal{S} \to \mathbb{Z}^{d_h}$ that maps each state to a discrete hash code:

$$\phi_h(s_t) \;=\; \text{round}\left(\frac{W^\top s_t}{\Delta}\right), \tag{3.5}$$

where $W \in \mathbb{R}^{d_s \times d_h}$ is a random projection matrix sampled once at initialization, d_h is the hash dimension, and $\Delta > 0$ is the bin width. Let $N(\phi_h(s_t))$ denote the number of times the hash code $\phi_h(s_t)$ has been visited prior to time step t. The raw intrinsic reward is the inverse-square-root visitation bonus

$$I_t \;=\; \frac{1}{\sqrt{\max\{N(\phi_h(s_t)),\, 1\}}}. \tag{3.6}$$

This module has no learnable parameters; the hash table $N(\cdot)$ is updated incrementally as new states are visited.

3.2.3 Rewarding Impact-Driven Exploration

In the RIDE instantiation [12], the raw intrinsic reward measures the magnitude of representation change between consecutive states in a learned embedding space, divided by an episodic visitation count. Like ICM, RIDE learns an encoder ϕ and auxiliary forward and inverse dynamics models f_η^F, f_η^I trained with the same combined loss $L_{\text{ICM}}(\eta)$ from Equation (3.3); unlike ICM, the forward prediction error is used *only* to train the encoder, never as the intrinsic signal. The raw intrinsic reward is instead the impact of the transition on the latent representation, attenuated by an episodic count:

$$y_t \;=\; \phi(s_t) \in \mathbb{R}^{d_y}, \tag{3.7}$$

$$I_t \;=\; \frac{\left\|\phi(s_{t+1}) - \phi(s_t)\right\|_2}{\sqrt{\max\{N_e(\phi_h(s_{t+1})),\, 1\}}}, \tag{3.8}$$

where ϕ_h is the same random-projection hash defined in Equation (3.5) and $N_e(\cdot)$ counts the number of visits to the corresponding hash code *within the current episode* (the count is reset at every episode boundary). The numerator rewards transitions whose representation change is large, capturing the intuition that meaningful actions move the agent through latent space rather than leaving it in place; the denominator suppresses bonuses for revisiting states already encountered in the same episode, which prevents the cyclical wandering that arises when prediction-based bonuses are used in procedurally generated environments. Because the encoder is updated by the same inverse-plus-forward objective as ICM, RIDE inherits ICM's representation learning while replacing the intrinsic signal with an episodic, impact-driven one. We use the same hash dimension $d_h = 32$ and bin width $\Delta = 0.1$ as in the count-based module, and an embedding dimension of $d_y = 256$ matching the ICM encoder.

3.2.4 Standardization and Rectification

Regardless of the module, the raw intrinsic reward I_t produced by Equation (3.4), (3.6), or (3.8) is non-stationary: its magnitude evolves during training as the underlying generator (forward model, hash counts, or learned encoder) adapts to the agent's experience. To keep the input to the Beta Network bounded in scale and approximately stationary across updates, the same preprocessing step is applied to all three modules.

Definition 3.3 (Standardized Intrinsic Reward)**.** Given a minibatch with empirical mean μ_I and standard deviation σ_I of the raw intrinsic rewards, and a small constant $\varepsilon > 0$, the *standardized intrinsic reward* is

$$I_t^+ \;=\; \max\!\left(0,\; \frac{I_t - \mu_I}{\sigma_I + \varepsilon}\right). \tag{3.9}$$

This preprocessing suppresses transitions whose intrinsic signal is below the batch average and retains relatively surprising transitions. We use the generic notation $I_t^{+,m}$ to emphasize that the same pipeline (standardization, Beta Network, progress loss) is applied identically across all three modules; only the upstream raw signal I_t differs.

3.3 Critic-Adaptive Reward Exploration

CARE introduces a state-dependent scaling function $\beta_\psi(s_t)$ between the intrinsic reward generator and the policy update. This section formalizes the network, its learning-progress training objective, and the reasons for choosing a first-order optimization scheme over a meta-gradient one.

3.3.1 Beta Network

Definition 3.4 (Beta Network)**.** The *Beta Network* is a parameterized map

$$\beta_\psi : \mathcal{S} \to [\beta_{\min}, \beta_{\max}], \qquad 0 < \beta_{\min} < \beta_{\max} < \infty, \tag{3.10}$$

which assigns a positive, bounded scaling factor to each state. In the implemented system, β_ψ is realized as a feed-forward MLP with a 2-layer encoder of width 256 and a 2-layer head of widths $[256, 128, 1]$ that produces $\log \beta_\psi(s_t)$. The output is mapped to $[\beta_{\min}, \beta_{\max}] = [0.001, 0.1]$ via $\beta_\psi(s_t) = \exp\big(\text{clamp}(\log \beta_\psi(s_t),\, \log \beta_{\min},\, \log \beta_{\max})\big)$. The complete architecture and weight initialization are listed in Appendix B.1, Table B.1.

For an intrinsic reward module m, the scaled intrinsic reward at time step t is

$$\tilde{I}_t^m \;=\; \beta_\psi(s_t)\, I_t^{+,m}, \tag{3.11}$$

and substituting Equation (3.11) into Equation (3.2) yields the shaped reward used by PPO.

The boundedness assumption built into Definition 3.4 ensures that the shaped reward cannot grow arbitrarily large relative to the extrinsic signal, regardless of the behavior of the Beta Network during training.

Lemma 3.1 (Bounded Shaped Reward)**.** *Let β_ψ satisfy Definition 3.4 and assume the standardized intrinsic reward is bounded almost surely, $|I_t^{+,m}| \leq M$ for some constant $M < \infty$. Then for every time step t,*

$$\left|\bar{r}_t - R_t^E\right| \;\leq\; \beta_{max}\, M.$$

Proof in Appendix A.3.

Lemma 3.1 is what allows CARE to be inserted into a PPO training loop without introducing additional instability: the shaped reward differs from the extrinsic reward by a bounded perturbation, so the policy gradient remains bounded under the same regularity conditions that PPO already requires.

3.3.2 Learning-Progress Objective

The Beta Network is trained so that $\beta_\psi(s_t)$ becomes large at states where the value function exhibits large temporal-difference (TD) error and small at states where the value function is well-fitted. The intuition is that a large $|\delta_t|$ marks a state where extrinsic learning is still in progress, so additional intrinsic exploration there is informative; conversely, when $|\delta_t|$ is small, the agent already has a stable value estimate and additional curiosity bonus would not improve learning.

For each transition in the rollout buffer, the one-step TD error of the extrinsic value function is

$$\delta_t \;=\; R_t^E \;+\; \gamma\, V_\mu(s_{t+1})(1 - d_t) \;-\; V_\mu(s_t), \tag{3.12}$$

where $d_t \in \{0, 1\}$ is the episode termination flag and V_μ is the current critic. The per-state target $\tau(s_t)$ is then defined as a piecewise normalization of $|\delta_t|$ onto the Beta Network's output range:

$$\tau(s_t) \;=\; \begin{cases} \beta_{\min} + (\beta_{\max} - \beta_{\min}) \dfrac{|\delta_t|}{\max_{t'} |\delta_{t'}| + \varepsilon}, & \text{if } \max_{t'} R_{t'}^E > 0, \\ \beta_0, & \text{otherwise (cold-start fallback).} \end{cases} \tag{3.13}$$

The cold-start fallback addresses the regime in which a batch contains no positive extrinsic reward; in that case $|\delta_t|$ reflects mainly initialization noise of the critic, and would be a poor target for β_ψ. Defaulting τ to the reference value β_0 (set to the geometric centre of $[\beta_{\min}, \beta_{\max}]$, i.e. $\beta_0 = 0.01$) in that regime makes CARE behave like a fixed-coefficient method until extrinsic signal becomes informative.

The learning-progress loss is the mean-squared deviation of β_ψ from this target:

$$L_{\text{progress}}(\psi) \;=\; \mathbb{E}\Big[\big(\beta_\psi(s_t) - \tau(s_t)\big)^2\Big]. \tag{3.14}$$

A log-space regularizer pulls β_ψ toward the reference value β_0 to prevent degenerate scaling and to control variance during early training:

$$L_{\text{reg}}(\psi) \;=\; \mathbb{E}\Big[\big(\log \beta_\psi(s_t) - \log \beta_0\big)^2\Big]. \tag{3.15}$$

The complete Beta-network objective is

$$L_\beta(\psi) \;=\; \gamma_p \, L_{\text{progress}}(\psi) \;+\; \lambda_{\text{reg}} \, L_{\text{reg}}(\psi), \tag{3.16}$$

with $\gamma_p > 0$ the progress weight and $\lambda_{\text{reg}} > 0$ the regularization weight.

The progress loss has a clean statistical interpretation, and its gradient has a clean directional one.

Proposition 3.1 (Target Recovery)**.** *Assume the Beta Network has sufficient expressive capacity to represent any measurable function $\mathcal{S} \to [\beta_{min}, \beta_{max}]$. In the population limit and with $\lambda_{reg} = 0$, the unique minimizer $\psi^\star$ of $L_\beta(\psi)$ satisfies*

$$\beta_{\psi^\star}(s) \;=\; \mathbb{E}\big[\,\tau(s_t) \,\big|\, s_t = s\big]$$

for every state s visited with positive probability. Proof in Appendix A.1.

Proposition 3.1 states that, in the population limit, the optimal Beta Network simply outputs the conditional mean of the target τ at each state. Because τ is a monotone normalization of $|\delta_t|$, this means that $\beta_{\psi^\star}(s)$ is large precisely at states where the extrinsic value function has, on average, larger TD error. The next result describes the gradient direction during training.

Proposition 3.2 (Gradient Direction)**.** *For every state $s \in \mathcal{S}$ visited with positive probability,*

$$\frac{\partial L_{progress}}{\partial \beta_\psi(s)} \;=\; 2\,\mathbb{P}(s_t = s)\,\Big(\beta_\psi(s) \;-\; \mathbb{E}[\tau(s_t) \mid s_t = s]\Big).$$

In particular, gradient descent on $L_{progress}$ moves $\beta_\psi(s)$ monotonically toward the conditional mean target $\mathbb{E}[\tau(s_t) \mid s_t = s]$. Proof in Appendix A.2.

Proposition 3.2 makes the central design intuition of CARE explicit: a gradient step adjusts $\beta_\psi(s)$ toward the local average of τ, which by Equation (3.13) is a normalized measure of how unsettled the value function still is at state s. The Beta Network is therefore supervised by the agent's own learning progress as observed through the critic's TD error.

3.3.3 First-Order Optimization vs. Meta-Gradient

A natural alternative to the learning-progress objective is to learn the intrinsic reward weighting via meta-gradients, as in Learning Intrinsic Rewards for Policy Gradient methods (LIRPG) [20]. In that approach, the parameters of the intrinsic reward function are updated so that, after a policy update using the shaped reward, the resulting policy maximizes the extrinsic objective. This requires differentiating through one or more inner policy updates, which is computationally expensive and can be unstable in deep reinforcement learning settings.

CARE replaces the meta-gradient by a first-order objective that supervises β_ψ directly through the target $\tau(s_t)$ derived from the critic's TD error. The cost difference is quantified by the following result.

Proposition 3.3 (Computational Cost)**.** *Per Beta-network update of size T, let K denote the number of inner policy update steps used in a meta-gradient estimator, $|\theta|$ the number of policy parameters, and $|\psi|$ the number of Beta-network parameters. Then a meta-gradient training step has worst-case time complexity $O(K\,T\,|\theta|)$, while a CARE first-order training step has time complexity $O(T\,|\psi|)$. In particular, when $|\psi| \ll |\theta|$ and $K \geq 1$, the CARE update is asymptotically cheaper by a factor of $K\,|\theta|/|\psi|$. Proof in Appendix A.4.*

In the implemented system, $|\psi|$ is roughly two orders of magnitude smaller than $|\theta|$, so the cost reduction is substantial. The price of this efficiency is a weaker form of supervision: instead of optimizing β_ψ directly for downstream extrinsic performance, CARE optimizes it for a TD-error-based target that serves as a proxy for that performance. The next result describes two regimes in which this proxy reduces to the trivial fixed-coefficient case.

Corollary 3.1 (Degeneration to Fixed Coefficient)**.** *Consider the objective* $L_\beta(\psi)$ *in Equation* (3.16)*. There are two regimes in which every minimizer* $\psi^\star$ *satisfies* $\beta_{\psi^\star}(s) \to \beta_0$ *uniformly in* s*, and the shaped reward in Equation* (3.2) *reduces to the fixed-coefficient form* $\bar{r}_t = R_t^E + \beta_0\, I_t^{+,m}$*:*

(i) ***Cold-start regime****: when* $\max_t R_t^E \le 0$ *within the rollout buffer, the target* τ *in Equation* (3.13) *reduces to* β_0 *by definition, and the unique minimizer of* $L_{progress}$ *is* $\beta_\psi \equiv \beta_0$*.*

(ii) ***Strong regularization regime****: as* $\lambda_{reg} \to \infty$*, the regularization term dominates and forces* $\log \beta_\psi(s) \to \log \beta_0$ *in* L^2 *under the state-visitation distribution, equivalently* $\beta_\psi(s) \to \beta_0$*.*

Proof in Appendix A.5.

Corollary 3.1 has both a theoretical and a practical reading. Theoretically, it shows that CARE strictly contains the fixed-coefficient baseline PPO + module m (with coefficient β_0) as a limiting case for any intrinsic reward module m, so the framework cannot perform worse than that baseline once the regularization is appropriately tuned and the cold-start fallback is in place. Practically, the cold-start branch is essential in extremely sparse environments: when the agent has not yet encountered any extrinsic reward, the TD error reflects only initialization noise of the critic, and falling back to β_0 prevents CARE from amplifying that noise into the policy update.

3.4 Algorithm

This section formalizes the data structures, loss functions, and training procedure used to optimize the system end to end.

3.4.1 Rollout Buffer

Each PPO update operates on an on-policy rollout buffer of length T collected by the current policy. A single transition is the tuple

$$\xi_t \;=\; \big(s_t,\; a_t,\; \log \pi_\theta(a_t \mid s_t),\; R_t^E,\; s_{t+1},\; d_t\big), \tag{3.17}$$

where $d_t \in \{0, 1\}$ is the episode termination flag. The buffer of length T is stored as the tensors

$$\mathcal{S}_{\text{tensor}} \in \mathbb{R}^{T \times d_s}, \quad \mathcal{A}_{\text{tensor}} \in \mathbb{N}^T, \quad \mathcal{R}_{\text{tensor}} \in \mathbb{R}^T, \quad \mathcal{S}'_{\text{tensor}} \in \mathbb{R}^{T \times d_s}, \quad \mathcal{D}_{\text{tensor}} \in \{0, 1\}^T,$$

together with the log-probabilities $\mathcal{P}_{\text{tensor}} \in \mathbb{R}^T$. Following the implementation defaults, $T = 4 \times \text{max\_ep\_len}$ steps are collected before each PPO update.

3.4.2 Loss Functions

Four distinct losses are minimized at each update cycle. They share the same rollout buffer but are optimized with respect to disjoint parameter groups, so no joint optimization is required.

PPO Actor Loss $L^{\mathbf{PPO}}(\theta)$. Define the importance ratio $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ and let $\hat{A}_t$ denote the advantage estimate computed by Generalized Advantage Estimation (GAE) [21] from the shaped reward $\bar{r}_t$. The clipped surrogate objective [22] is

$$L^{\text{PPO}}(\theta) \;=\; -\,\mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\,\hat{A}_t,\; \text{clip}\big(\rho_t(\theta),\, 1-\epsilon,\, 1+\epsilon\big)\,\hat{A}_t\Big)\Big]. \tag{3.18}$$

Critic Loss $L^V(\mu)$. The value network V_μ is fitted by mean-squared error against the GAE return target $\hat{V}_t$:

$$L^V(\mu) \;=\; \mathbb{E}_t\Big[\big(V_\mu(s_t) - \hat{V}_t\big)^2\Big]. \tag{3.19}$$

ICM Loss $L_{\mathbf{ICM}}(\eta)$. As specified in Equation (3.3).

Beta Loss $L_\beta(\psi)$. As specified in Equation (3.16).

The four losses correspond to four parameter groups: θ (actor), μ (critic), η (ICM), and ψ (Beta Network). Gradients flow through each loss with respect to its own parameter group only; in particular, the Beta Network is not updated through L^{PPO} and the policy is not updated through L_β.

Target Beta Network. To decouple the β used for reward shaping from the β currently being optimized, the implementation maintains a *target* Beta Network $\bar{\beta}_{\bar{\psi}}$ alongside the live network β_ψ. After each ψ-step, the target is soft-updated via a Polyak average:

$$\bar{\psi} \;\leftarrow\; (1 - \tau_{\text{target}})\,\bar{\psi} \;+\; \tau_{\text{target}}\,\psi, \tag{3.20}$$

with $\tau_{\text{target}} = 0.01$. The shaped reward in Equation (3.2) uses $\bar{\beta}_{\bar{\psi}}(s_t)$ (the lagged target) rather than $\beta_\psi(s_t)$ (the live network). This breaks the within-update circular dependency: δ_t is derived from V_μ shaped by the *previous* target $\bar{\psi}$, so it is not influenced by the ψ update occurring in the same iteration.

3.4.3 Training Procedure

Algorithm 1 specifies the complete CARE training loop. One iteration of the outer loop corresponds to a single PPO update cycle.

Algorithm 1 CARE training loop with PPO and an intrinsic reward module.

1: Initialize policy θ, critic μ, intrinsic-module parameters η (if any), Beta Network ψ
2: **for** update $= 1, 2, \ldots, U$ **do**
3: Collect T on-policy transitions $\{\xi_t\}_{t=0}^{T-1}$ using π_θ
4: Compute raw intrinsic rewards I_t via the module-specific equation in Section 3.2
5: Standardize and rectify to obtain I_t^+ via Equation (3.9)
6: Update intrinsic-module parameters η (no-op if module is non-parametric, e.g., count-based)
7: Compute extrinsic TD errors δ_t via Equation (3.12) and targets $\tau(s_t)$ via Equation (3.13)
8: Update Beta Network: $\psi \leftarrow \psi - \psi_{\text{lr}} \nabla_\psi L_\beta(\psi)$
9: Compute shaped rewards $\bar{r}_t = R_t^E + \beta_\psi(s_t)\, I_t^+$
10: Compute GAE advantages $\hat{A}_t$ from $\{\bar{r}_t\}$ and V_μ
11: **for** epoch $= 1, 2, \ldots, K$ **do**
12: Update actor: $\theta \leftarrow \theta - \theta_{\text{lr}} \nabla_\theta L^{\text{PPO}}(\theta)$
13: Update critic: $\mu \leftarrow \mu - \mu_{\text{lr}} \nabla_\mu L^V(\mu)$
14: **end for**
15: **end for**

The algorithm preserves the standard PPO update structure (lines 11–14) and inserts only two additional optimization steps (lines 6 and 8) before the shaped reward is computed. This modularity is important: the same template applies if ICM is replaced by a count-based bonus or by RIDE, with only Equations (3.4)–

(3.9) being substituted by the corresponding module-specific computation. Concrete numerical values for all hyperparameters and network architectures used in the experiments are listed in Appendix B.

Chapter 4

Experimental Setup and Results

This chapter describes the experimental setup used to evaluate the proposed methodology. It first presents the benchmark environments and the computing infrastructure, then specifies the experimental configuration, the baselines, and the performance evaluation metrics. The empirical evaluation across all three intrinsic reward families (count-based, ICM, and RIDE) is the subject of Section 4.6. Settings reported here are the canonical configuration shared by every reported run; the underlying hyperparameter values are tabulated in Appendix B.

4.1 Benchmark Environments

The empirical evaluation is conducted on six sparse-reward MiniGrid environments [16]. These environments are chosen because they cover qualitatively different forms of exploration difficulty while sharing a common gridworld interface, which keeps the comparison across intrinsic reward modules controlled.

Bång 4.1: MiniGrid environments used in the experiments.

Environment	**Purpose in the evaluation**
DoorKey-8x8	Tests whether the agent can discover a key, unlock a door, and shift from exploration to goal-directed behavior.
Empty-16x16	Represents an extreme sparse-reward setting with almost no intermediate feedback before the goal is reached.
KeyCorridor-S3R3	A multi-room layout with several keys and locked doors; assesses sustained exploration over a longer horizon.
LavaCrossing-S9N3	Requires safe navigation around lava; penalizes irrelevant exploration that leads to early termination.
RedBlueDoors-8x8	Requires opening two doors in the correct order, testing whether the method can suppress irrelevant exploration.
UnlockPickup	Extends DoorKey with an additional object manipulation stage, creating a hierarchical sparse-reward task.

These tasks share several useful properties for studying adaptive intrinsic reward scaling: sparse terminal rewards, egocentric partial observations, step penalties that discourage random motion, and stochastic initialization across random seeds. Together, they form a test bed in which different intrinsic reward modules can be compared under the same policy optimization backbone and the same observation interface.

4.2 Computing Environment

All experiments were carried out on a single machine with the following configuration:

- **Hardware**: [CPU model, e.g., Intel Core i7-13700K, 16 cores]; [GPU model, e.g., NVIDIA RTX 4070, 12 GB VRAM]; [RAM, e.g., 32 GB].
- **Operating system**: Windows 11.
- **Software stack**: Python 3.12, PyTorch with CUDA support, `gymnasium` with the `minigrid` extension, `numpy`, `pandas`, and `matplotlib`.
- **Wall-clock budget**: each seeded run consists of 10^6 environment steps; the approximate runtime per seed is [fill in after benchmarking] hours on the configuration above.

The training scripts automatically dispatch the policy and auxiliary networks to GPU when CUDA is available and fall back to CPU otherwise. All experiments use deterministic random seeds for reproducibility, and all training logs and checkpoint files are saved to disk every 10^5 steps.

4.3 Experimental Configuration

All reported runs share the same PPO backbone and the same intrinsic-reward processing pipeline so that the comparison across modules and across scaling strategies is fair. The complete set of hyperparameter values is tabulated in Appendix B.2; the most relevant high-level settings are summarized here.

No separate global intrinsic strength coefficient is used: the Beta Network output $\beta_\psi(s_t)$ alone determines the intrinsic-reward scale, and in fixed-β conditions the same scalar β plays this role. In the CARE conditions, the Beta Network is realized as a two-layer encoder of dimension 256 followed by a 128-unit head producing $\log \beta_\psi$, which is clipped so that $\beta_\psi(s_t) \in [\beta_{\min}, \beta_{\max}] = [0.001, 0.1]$. This range is chosen to span an order of magnitude either side of the optimal fixed coefficients reported for sparse MiniGrid tasks (approximately $\beta \approx 0.005$ for count-based bonuses and $\beta \approx 0.05$ for ICM [**?**]), so that the adaptive variant has the freedom to recover, undershoot, or overshoot any of those values. The learning-progress objective in Equation (3.16) uses regularization weight $\lambda_{\text{reg}} = 10^{-3}$, reference value $\beta_0 = 0.01$ (the geometric centre of the interval), gradient clipping at L_2 norm 1.0, and L_2 weight decay of 10^{-6} on the Beta Network parameters. The Beta Network is updated with Adam at learning rate 5×10^{-4}. In the fixed-β conditions, the

Beta Network is replaced by a constant scalar $\beta \in \{0.001, 0.005, 0.05, 0.1\}$ and the learning-progress update is disabled; the rest of the pipeline (intrinsic generation, standardization, and the shaped reward formula in Equation (3.2)) is unchanged.

The base PPO algorithm uses $K = 80$ epochs per update, clip parameter $\epsilon = 0.2$, discount factor $\gamma = 0.99$, GAE bias-variance trade-off $\lambda = 0.95$, actor learning rate 3×10^{-4}, and critic learning rate 10^{-3}. The rollout buffer length is $T = 4{,}000$ steps, and each seed is trained for 10^6 environment steps. Each configuration is repeated over five independent random seeds $\{1, 2, 3, 4, 5\}$.

4.4 Baselines

The experimental framing of this thesis treats CARE as a module-agnostic adaptive layer that can be paired with any of the three representative intrinsic reward families: count-based bonuses, the Intrinsic Curiosity Module (ICM), and Rewarding Impact-Driven Exploration (RIDE). To isolate the contribution of CARE from both the contribution of the underlying intrinsic signal and the choice of any particular fixed scaling coefficient, the baselines are organized into a $1 + 12 + 3$ matrix as summarized in Table 4.2: one no-intrinsic control, twelve fixed-β configurations spanning a two-order-of-magnitude sweep across the three modules, and three CARE variants.

Bång 4.2: Baseline conditions evaluated on every environment.

Label	**Configuration**	**Purpose in the comparison**
B_0	PPO (no intrinsic reward)	Lower bound on exploration ability without any curiosity signal.
$F_{m,\beta}$	PPO + module m with constant β, $m \in \{\text{Count}, \text{ICM}, \text{RIDE}\}$, $\beta \in \{0.001, 0.005, 0.05, 0.1\}$	Fixed-coefficient sweep for each module. The four values span two orders of magnitude and bracket the optimal coefficients reported for sparse MiniGrid tasks ($\beta \approx 0.005$ for count-based and $\beta \approx 0.05$ for ICM [**?**]).
C_{Count}	PPO + count-based + CARE	Count-based bonus with learned $\beta_\psi(s_t)$ from the Beta Network.
C_{ICM}	PPO + ICM + CARE	ICM bonus with learned $\beta_\psi(s_t)$.
C_{RIDE}	PPO + RIDE + CARE	RIDE bonus with learned $\beta_\psi(s_t)$.

This design isolates three distinct effects:

- B_0 **vs** $F_{m,\beta}$ measures whether intrinsic motivation in module m at coefficient β helps or hurts on top of the extrinsic-only PPO baseline. Sweeping β over two orders of magnitude around the literature-reported optima exposes how sensitive each module is to manual tuning within the relevant regime.

- $F_{m,\beta}$ **vs** C_m measures the contribution of CARE's adaptive scaling, holding the underlying intrinsic signal fixed. The cleanest test is the comparison of C_m against the *best* fixed-β value for module m on each environment: if CARE matches or exceeds the best post-hoc fixed coefficient *without* that coefficient having to be searched, the case for state-dependent scaling is established.

- $C_{\textbf{Count}}$ **vs** $C_{\textbf{ICM}}$ **vs** $C_{\textbf{RIDE}}$ reveals how the same CARE layer interacts with structurally different intrinsic generators (count-based visitation, ICM forward error, and RIDE impact-with-episodic-counts), addressing the research question of module-agnosticism stated in Chapter 1.

With $1 + 12 + 3 = 16$ conditions per environment and five seeds per condition, the full evaluation comprises $16 \times 5 = 80$ runs per environment and $80 \times 6 = 480$ runs across the six environments.

4.5 Performance Evaluation Metrics

Four metrics are used to quantify and compare the conditions defined above.

Episode return. Let R_t^E be the extrinsic reward at time t in evaluation episode e. The episode return for that episode is $\mathcal{R}_e = \sum_t R_t^E$. The reported curves are the mean of $\mathcal{R}_e$ over a sliding window of evaluation episodes around each checkpoint, averaged across the five seeds. Episode return is the primary metric of task success.

Sample efficiency. For each environment, a target return $r^\star$ is fixed at a fraction (typically 0.7) of the asymptotic return achieved by the strongest baseline. The sample efficiency of a method is then the smallest number of environment steps $n^\star$ at which its mean evaluation return first reaches $r^\star$:

$$n^\star \;=\; \min\bigl\{n : \overline{\mathcal{R}}(n) \geq r^\star\bigr\},$$

with $n^\star = +\infty$ if the threshold is never reached within the 10^6-step budget. Lower values of $n^\star$ indicate faster learning.

Learning stability. For each method and each timestep n, the learning stability is reported as the across-seed standard deviation $\sigma(n)$ of the mean evaluation return at that timestep:

$$\sigma(n) \;=\; \sqrt{\frac{1}{|S| - 1} \sum_{s \in S} \bigl(\overline{\mathcal{R}}_s(n) - \overline{\mathcal{R}}(n)\bigr)^2},$$

where S is the set of seeds. Lower $\sigma(n)$ indicates more reproducible behavior.

Learned scaling behavior. The learned function β_ψ is analyzed both globally and locally. Globally, the empirical distribution of $\beta_\psi(s_t)$ over the rollout buffer is summarized by its mean and selected percentiles $(25\%, 50\%, 75\%)$ at fixed checkpoints throughout training. Locally, per-environment heatmaps of $\beta_\psi(s)$ projected onto the agent's two-dimensional position are produced from the trajectory data saved every 10^5 steps, providing a qualitative view of where in state space CARE allocates more exploration influence.

4.6 Empirical Results

Empirical results across the six benchmark environments and the sixteen conditions defined in Section 4.4 will be reported here once the experimental runs are complete. The remainder of this section is organized into the following subsections, which will be populated with reward curves, summary tables, and qualitative analyses of $\beta_\psi(s)$.

4.6.1 Module-Level Results: Fixed-β Sweep vs CARE

Per-module reward curves and summary statistics. For each module $m \in \{Count, ICM, RIDE\}$, *the five* $F_{m,\beta}$ *curves and the* C_m *curve are plotted on a single panel per environment, with* B_0 *as a shared reference. Tables report the best fixed-*β *value per environment and the gap to* C_m.

4.6.2 Effect of CARE Across Modules

Cross-module comparison summarizing where CARE provides the largest gain over the best fixed coefficient, where it is neutral, and where the cold-start fallback dominates. Includes a heatmap of CARE-vs-best-fixed gains across the six environments and three modules.

4.6.3 Sensitivity to Fixed β

Analysis of how sharply performance varies across the swept range $\beta \in \{0.001, 0.005, 0.05, 0.1\}$ *for each module, motivating the need for adaptive scaling: modules whose performance varies strongly with* β *are the ones where CARE is expected to help most.*

4.6.4 Behavior of the Learned Scaling Function

Distributional and trajectory-level analysis of $\beta_\psi(s_t)$, *including evolution over training and qualitative state heatmaps.*

4.6.5 Discussion

Synthesis of the empirical findings in light of the formal results in Chapter 3 and the literature reviewed in Chapter 2.

Conclusion and Future Work

This thesis studies Critic-Adaptive Reward Exploration (CARE) in reinforcement learning. Instead of treating CARE as a method used with only one type of curiosity reward, this work views it as an adaptive layer to study how state-dependent scaling of intrinsic rewards affects learning. The thesis considers three representative intrinsic reward families: count-based bonuses, the Intrinsic Curiosity Module (ICM), and Rewarding Impact-Driven Exploration (RIDE). CARE is used as the mechanism that decides how much these rewards should influence learning in each state. By scaling intrinsic reward in proportion to the magnitude of the value function's temporal-difference error at each state, CARE offers a simple way to balance exploration and exploitation during training without using costly meta-gradient methods.

Experiments on MiniGrid evaluate each intrinsic reward module under a sweep of fixed scaling coefficients $\beta \in \{0.001, 0.005, 0.05, 0.1\}$ and under the CARE adaptive variant, with PPO without intrinsic reward as a control. The results show that CARE improves learning stability and sample efficiency in several sparse-reward tasks, especially when extrinsic rewards are rare but still useful. Compared to manually tuned fixed coefficients, the adaptive approach works more consistently across different environments and random seeds, often matching or exceeding the best post-hoc fixed value of β without requiring an environment-specific search. However, the results also show a limitation: when extrinsic rewards are almost always missing, the temporal-difference signal that drives the learning-progress target becomes uninformative, and the scaling factor falls back to its reference value β_0, behaving like a fixed coefficient. This suggests that the effectiveness of CARE depends on how often the agent encounters extrinsic feedback that produces meaningful TD error.

There are several directions for future work. First, the method can be applied to broader settings such as continuous control, multi-task RL, and embodied navigation, where the structure of intrinsic motivation differs from the gridworld setting studied here. Second, a clearer theoretical analysis of the learning-progress objective could help explain when adaptive scaling is most useful across different intrinsic reward types. Third, alternative target signals for the Beta Network be-

yond TD-error magnitude—for example bootstrapped novelty or trajectory-level success indicators—may improve robustness in extremely sparse regimes where the current target degenerates. These directions can further improve the use of adaptive intrinsic rewards in reinforcement learning.

Tài liệu tham khảo

[1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[2] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

[3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

[4] Vincent François-Lavet, Peter Henderson, Riashat Islam, Marc G. Bellemare, and Joelle Pineau. An introduction to deep reinforcement learning. *Foundations and Trends in Machine Learning*, 11(3–4):219–354, 2018.

[5] Pierre-Yves Oudeyer, Frédéric Kaplan, and Verena V. Hafner. Intrinsic motivation systems for autonomous mental development. *IEEE Transactions on Evolutionary Computation*, 11(2):265–286, 2007.

[6] Jürgen Schmidhuber. Formal theory of creativity, fun, and intrinsic motivation. *IEEE Transactions on Autonomous Mental Development*, 2(3):230–247, 2010.

[7] Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Rémi Munos. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems*, volume 29, pages 1471–1479, 2016.

[8] Haoran Tang, Rein Houthooft, David Foote, Adam Stooke, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. #exploration: A study of count-based exploration for deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 30, pages 2753–2762, 2017.

[9] Georg Ostrovski, Marc G. Bellemare, Aaron van den Oord, and Rémi Munos. Count-based exploration with neural density models. In *Proceedings of the 34th*

International Conference on Machine Learning, volume 70, pages 2721–2730, 2017.

[10] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 2778–2787, 2017.

[11] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations*, 2019.

[12] Roberta Raileanu and Tim Rocktäschel. Ride: Rewarding impact-driven exploration for procedurally-generated environments. In *International Conference on Learning Representations*, 2020.

[13] Younggyo Seo, Lili Chen, Jinwoo Shin, Honglak Lee, Pieter Abbeel, and Kimin Lee. State entropy maximization with random encoders for efficient exploration. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pages 9443–9454, 2021.

[14] Eric Chen, Zhang-Wei Hong, Joni Pajarinen, and Pulkit Agrawal. Redeeming intrinsic rewards via constrained optimization. In *Advances in Neural Information Processing Systems*, volume 35, pages 24041–24054, 2022.

[15] Minghao Yuan, Bowen Li, Xiaoming Jin, and Wei Zeng. Automatic intrinsic reward shaping for exploration in deep reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 40356–40375, 2023.

[16] Maxime Chevalier-Boisvert, Lucas Willems, and Suman Pal. Minimalistic gridworld environment for openai gym. In *ICLR Workshop on Representation Learning for RL*, 2018.

[17] Arthur Aubret, Laetitia Matignon, and Salima Hassas. An information-theoretic perspective on intrinsic motivation in reinforcement learning: A survey. *Entropy*, 25(2):327, 2023.

[18] Adrià Puigdomènech Badia, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo, Charles Blundell, Timothy Lillicrap, and Bilal Piot. Never give up: Learn-

ing directed exploration strategies. In *International Conference on Learning Representations*, 2020.

[19] Adrià Puigdomènech Badia, Bilal Hu, Julian Schrittwieser, Pablo Sprechmann, Alex Vitvitskyi, Daniel Guo, Charles Blundell, Timothy Lillicrap, and David Silver. Agent57: Outperforming the atari human benchmark. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pages 507–517, 2020.

[20] Zeyu Zheng, Junhyuk Oh, and Satinder Singh. On learning intrinsic rewards for policy gradient methods. In *Advances in Neural Information Processing Systems*, volume 31, pages 4644–4654, 2018.

[21] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

[22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Appendix A

Mathematical Proofs

This appendix collects the full proofs of the formal results stated in Chapter 3. Each section below corresponds to a single proposition, lemma, or corollary, and may be read independently of the others. Throughout the appendix, expectations are taken with respect to the on-policy distribution induced by the current policy π_θ unless stated otherwise; standard regularity conditions (existence of first and second moments, measurability of β_ψ, and a state-visitation distribution that admits a density) are assumed.

A.1 Proof of Proposition 3.1: Target Recovery

Chứng minh. We work with the population form of the learning-progress loss in Equation (3.14):

$$L_{\text{progress}}(\psi) \;=\; \mathbb{E}\Big[\big(\beta_\psi(s_t) - \tau(s_t)\big)^2\Big],$$

where the expectation is over the on-policy state-visitation distribution. Setting $\lambda_{\text{reg}} = 0$ in Equation (3.16) reduces the Beta-network objective to $L_\beta(\psi) = \gamma_p\, L_{\text{progress}}(\psi)$, which is minimized at the same ψ as L_{progress}.

Assume the Beta Network has sufficient capacity to represent any measurable function $f : \mathcal{S} \to [\beta_{\min}, \beta_{\max}]$. Then minimizing L_{progress} over ψ is equivalent to minimizing

$$\mathcal{J}(f) \;=\; \mathbb{E}\Big[\big(f(s_t) - \tau(s_t)\big)^2\Big]$$

over the function class. Conditioning on s_t and applying the law of total expectation,

$$\mathcal{J}(f) \;=\; \mathbb{E}\big[\mathbb{E}\big[(f(s_t) - \tau(s_t))^2 \,\big|\, s_t\big]\big].$$

Because $f(s_t)$ is constant given s_t, the inner expectation is the conditional mean-squared error of $f(s_t)$ as a predictor of $\tau(s_t)$. By the standard L^2-projection argument, this inner expectation is uniquely minimized (over choices of $f(s_t)$) by the conditional mean

$$f^\star(s) \;=\; \mathbb{E}\big[\,\tau(s_t) \,\big|\, s_t = s\big].$$

Since $\tau(s_t) \in [\beta_{\min}, \beta_{\max}]$ almost surely (by Equation (3.13) together with $\beta_0 \in [\beta_{\min}, \beta_{\max}]$), the conditional mean $f^\star(s)$ also lies in $[\beta_{\min}, \beta_{\max}]$, and is therefore representable by the Beta Network. Setting $\beta_{\psi^\star}(s) = f^\star(s)$ yields a global minimizer of L_β, and the strict convexity of the squared loss makes it unique on the support of the state-visitation distribution. □

A.2 Proof of Proposition 3.2: Gradient Direction

Chứng minh. Fix a state $s \in \mathcal{S}$ visited with positive probability, and treat $\beta_\psi(s)$ as a free real-valued parameter for the purpose of computing the partial derivative. The population form of the progress loss can be written as

$$L_{\text{progress}}(\psi) \;=\; \mathbb{E}\Big[\big(\beta_\psi(s_t) - \tau(s_t)\big)^2\Big] \;=\; \int_{\mathcal{S}} p(s')\, \mathbb{E}\big[(\beta_\psi(s') - \tau(s_t))^2 \,\big|\, s_t = s'\big]\, \mathrm{d}s',$$

where $p(s')$ denotes the density of the state-visitation distribution. Differentiating with respect to $\beta_\psi(s)$, only the term at $s' = s$ contributes,

$$\begin{aligned}
\frac{\partial L_{\text{progress}}}{\partial \beta_\psi(s)} \;&=\; p(s)\, \frac{\partial}{\partial \beta_\psi(s)}\, \mathbb{E}\big[(\beta_\psi(s) - \tau(s_t))^2 \,\big|\, s_t = s\big] \\
&=\; p(s)\, \mathbb{E}\big[2\,(\beta_\psi(s) - \tau(s_t)) \,\big|\, s_t = s\big] \\
&=\; 2\,p(s)\, \Big(\beta_\psi(s) - \mathbb{E}[\tau(s_t) \mid s_t = s]\Big).
\end{aligned}$$

Identifying $\mathbb{P}(s_t = s) \equiv p(s)$ in the discrete or density-weighted sense, this gives the announced identity

$$\frac{\partial L_{\text{progress}}}{\partial \beta_\psi(s)} \;=\; 2\,\mathbb{P}(s_t = s)\, \Big(\beta_\psi(s) - \mathbb{E}[\tau(s_t) \mid s_t = s]\Big).$$

In particular, the partial derivative is positive when $\beta_\psi(s)$ overshoots the conditional mean target and negative when it undershoots, so a gradient descent step on L_{progress} shrinks $\beta_\psi(s)$ in the first case and grows it in the second. Repeated application of this update therefore drives $\beta_\psi(s)$ monotonically toward $\mathbb{E}[\tau(s_t) \mid s_t = s]$ until equality is reached, in agreement with Proposition 3.1. □

A.3 Proof of Lemma 3.1: Bounded Shaped Reward

Chứng minh. By Equation (3.2),

$$\bar{r}_t - R_t^E \;=\; \beta_\psi(s_t)\, I_t^{+,m}.$$

Taking absolute values,

$$\left|\bar{r}_t - R_t^E\right| \;=\; \left|\beta_\psi(s_t)\right| \left|I_t^{+,m}\right|.$$

By Definition 3.4, $\beta_\psi(s_t) \in [\beta_{\min}, \beta_{\max}]$ with $\beta_{\min} > 0$, so $|\beta_\psi(s_t)| \le \beta_{\max}$. By assumption, $|I_t^{+,m}| \le M$ almost surely. Combining these bounds,

$$\left|\bar{r}_t - R_t^E\right| \;\le\; \beta_{\max} M,$$

which establishes the claim. □

A.4 Proof of Proposition 3.3: Computational Cost Comparison

Chứng minh. We measure cost in scalar floating-point operations and ignore lower-order terms.

Meta-gradient training step. A meta-gradient training step for a parameterized intrinsic reward function (LIRPG [20]) consists of the following operations on a buffer of T transitions: (i) compute the policy gradient at the current parameters and apply K inner update steps to obtain a perturbed policy θ', each of which costs $O(T\,|\theta|)$ to evaluate the policy gradient and $O(|\theta|)$ to apply the update; and (ii) backpropagate the meta-objective evaluated at θ' through the chain of K inner updates to obtain a gradient with respect to the intrinsic reward parameters. The backward pass through K unrolled updates costs $O(K\,T\,|\theta|)$ in time and $O(K\,|\theta|)$ in memory because each inner step must be cached. Summing the forward and backward costs, the worst-case time complexity per buffer update is

$$C_{\text{meta}} \;=\; O(K\,T\,|\theta|).$$

CARE training step. A CARE training step for the Beta Network on the same buffer of T transitions consists of: (i) one forward pass through the critic V_μ for each of the T states to obtain TD errors δ_t, at cost $O(T\,|\mu|)$, where $|\mu| \le |\theta|$; (ii) construction of the target $\tau(s_t)$ from $|\delta_t|$, which is $O(T)$; (iii) one forward pass through β_ψ for each of the T states, at cost $O(T\,|\psi|)$; and (iv) one backward pass through β_ψ to compute $\nabla_\psi L_\beta$, at cost $O(T\,|\psi|)$. The dominant term is $O(T\,|\psi|)$ when $|\psi| \ge |\mu|$ in practice; otherwise the bound becomes $O(T\,|\mu|)$, which is still strictly

smaller than C_{meta} by a factor of K. We summarize the time complexity per buffer update as

$$C_{\text{CARE}} \;=\; O(T\,|\psi|).$$

Asymptotic ratio. Taking the ratio,

$$\frac{C_{\text{meta}}}{C_{\text{CARE}}} \;=\; \Theta\!\left(\frac{K\,|\theta|}{|\psi|}\right).$$

When $|\psi| \ll |\theta|$ and $K \geq 1$, this ratio is large, which is what the proposition asserts. □

A.5 Proof of Corollary 3.1: Degeneration to Fixed Coefficient

Chứng minh. We treat the two regimes separately.

(i) Cold-start regime. Suppose $\max_t R_t^E \leq 0$ within the rollout buffer. By Equation (3.13), the target reduces to the constant value $\tau(s_t) \equiv \beta_0$ for every t. Substituting into the progress loss in Equation (3.14),

$$L_{\text{progress}}(\psi) \;=\; \mathbb{E}\big[(\beta_\psi(s_t) - \beta_0)^2\big]\,.$$

This squared expectation is non-negative and equals zero if and only if $\beta_\psi(s) = \beta_0$ almost everywhere with respect to the state-visitation distribution. The regularizer L_{reg} in Equation (3.15) is also non-negative and is minimized at the same point, since $L_{\text{reg}}(\psi) = \mathbb{E}[(\log \beta_\psi(s_t) - \log \beta_0)^2]$ vanishes precisely when $\beta_\psi(s) = \beta_0$. Therefore both terms of L_β in Equation (3.16) are simultaneously minimized at $\beta_{\psi^\star}(s) \equiv \beta_0$.

(ii) Strong regularization regime. The progress loss L_{progress} is non-negative and bounded above by $(\beta_{\max} - \beta_{\min})^2$ since both $\beta_\psi(s_t)$ and $\tau(s_t)$ take values in $[\beta_{\min}, \beta_{\max}]$. The regularizer L_{reg} is non-negative. Let $\psi_\lambda^\star$ be any minimizer of L_β at regularization weight $\lambda \equiv \lambda_{\text{reg}}$. By optimality, comparing against any reference ψ_0 that satisfies $\beta_{\psi_0}(s) \equiv \beta_0$ (for which $L_{\text{reg}}(\psi_0) = 0$),

$$\gamma_p\, L_{\text{progress}}(\psi_\lambda^\star) \;+\; \lambda\, L_{\text{reg}}(\psi_\lambda^\star) \;\leq\; \gamma_p\, L_{\text{progress}}(\psi_0) \;+\; 0.$$

Rearranging,

$$\lambda\, L_{\text{reg}}(\psi_\lambda^\star) \;\leq\; \gamma_p \left(L_{\text{progress}}(\psi_0) - L_{\text{progress}}(\psi_\lambda^\star)\right) \;\leq\; \gamma_p\,(\beta_{\max} - \beta_{\min})^2.$$

Dividing by $\lambda > 0$,

$$L_{\text{reg}}(\psi^\star_\lambda) \;\le\; \frac{\gamma_p\,(\beta_{\max} - \beta_{\min})^2}{\lambda} \;\xrightarrow[\lambda\to\infty]{}\; 0.$$

Hence $\mathbb{E}[(\log \beta_{\psi^\star_\lambda}(s_t) - \log \beta_0)^2] \to 0$, which implies $\log \beta_{\psi^\star_\lambda}(s_t) \to \log \beta_0$ in L^2 under the state-visitation distribution. Equivalently, $\beta_{\psi^\star_\lambda}(s) \to \beta_0$ in L^2 on the support of that distribution.

Reduction to fixed-coefficient form. In either regime, substituting $\beta_\psi(s) = \beta_0$ into Equation (3.2) yields

$$\bar{r}_t \;=\; R^E_t \;+\; \beta_0\, I^{+,m}_t,$$

which is exactly the fixed-coefficient PPO update with intrinsic coefficient β_0. $\square$

Appendix B

Implementation Details

This appendix records the network architectures, hyperparameters, and training schedule used in the experiments reported in Chapter 3 and Chapter 4. All values reflect the canonical settings adopted across the six benchmark MiniGrid environments and are extracted from the project source code.

B.1 Network Architectures

Bång B.1: Beta Network β_ψ architecture.

Component	**Specification**
Input	state s_t flattened to $\mathbb{R}^{d_s}$
Encoder layer 1	Linear($d_s \to 256$), Tanh
Encoder layer 2	Linear($256 \to 256$), Tanh
Head layer 1	Linear($256 \to 128$), Tanh
Head output	Linear($128 \to 1$) producing $\log \beta_\psi(s_t)$
Output bound	$\beta_\psi(s_t) = \exp\big(\text{clamp}(\log \beta_\psi, \log \beta_{\min}, \log \beta_{\max})\big)$
$[\beta_{\min}, \beta_{\max}]$	[0.001, 0.1]
Encoder weight init	$\mathcal{N}(0,\, 1/\sqrt{d_{\text{in}}})$
Output bias init	$\log(0.01) \approx -4.605$ (so $\beta_\psi \approx 0.01$ at initialization, near the geometric c

Bång B.2: Intrinsic Curiosity Module (ICM) architecture.

Component	**Specification**
Encoder ϕ layer 1	Linear($d_s \to 256$), Tanh
Encoder ϕ layer 2	Linear($256 \to 256$), Tanh
Latent dimension	256
Forward model f_η^F	Linear($256 + \lvert\mathcal{A}\rvert \to 256$), Tanh, Linear($256 \to 256$)
Inverse model f_η^I	Linear($512 \to 256$), Tanh, Linear($256 \to \lvert\mathcal{A}\rvert$)
Forward weight α_F in L_{ICM}	0.2
Inverse weight α_I in L_{ICM}	0.8
Action representation in forward model	one-hot $\in \mathbb{R}^{\lvert\mathcal{A}\rvert}$
Encoder gradient flow	inverse loss only (forward loss detached)

Bång B.3: RIDE architecture. The encoder, forward model, and inverse model share the ICM design and training objective; the difference is the intrinsic reward formula in Equation (3.8), which uses representation impact divided by an episodic count rather than forward prediction error.

Component	**Specification**
Encoder ϕ layer 1	Linear($d_s \to 256$), Tanh
Encoder ϕ layer 2	Linear($256 \to 256$), Tanh
Latent dimension d_y	256
Forward model f_η^F	Linear($256 + \lvert\mathcal{A}\rvert \to 256$), Tanh, Linear($256 \to 256$)
Inverse model f_η^I	Linear($512 \to 256$), Tanh, Linear($256 \to \lvert\mathcal{A}\rvert$)
Episodic-count hash dim d_h	32
Episodic-count bin width Δ	0.1
Episodic count reset	At every episode boundary (via `is_terminals` buffer)
Forward/inverse weighting	$\alpha_F = 0.2$, $\alpha_I = 0.8$ (same as ICM)
Intrinsic signal	$I_t = \lVert\phi(s_{t+1}) - \phi(s_t)\rVert_2 / \sqrt{\max\{N_e(s_{t+1}), 1\}}$

Bång B.4: Actor and critic network architectures used by PPO.

Component	**Specification**
Actor body	Linear($d_s \to 64$), Tanh, Linear($64 \to 64$), Tanh
Actor head	Linear($64 \to \lvert\mathcal{A}\rvert$) producing logits; sampling via Categorical(logits)
Critic body	Linear($d_s \to 64$), Tanh, Linear($64 \to 64$), Tanh
Critic head	Linear($64 \to 1$)

B.2 Hyperparameters

Bång B.5: PPO hyperparameters.

Symbol	**Description**	**Value**
K	Number of PPO gradient epochs per update	80
ϵ	PPO clip parameter	0.2
γ	Discount factor	0.99
λ	GAE bias-variance trade-off	0.95
θ_{lr}	Actor learning rate	3×10^{-4}
μ_{lr}	Critic learning rate	1×10^{-3}
Entropy coefficient	Weight on $-H[\pi_\theta]$ in PPO loss	0.01
Value loss coefficient	Weight on critic loss in joint update	0.5

Bång B.6: ICM hyperparameters.

Symbol	**Description**	**Value**
η_{lr}	ICM learning rate	1×10^{-3}
ICM epochs	Number of gradient epochs per buffer update	4
ICM batch size	Mini-batch size within each epoch	64

Bång B.7: RIDE hyperparameters.

Symbol	**Description**	**Value**
$\eta_{\text{lr}}^{\text{RIDE}}$	RIDE encoder learning rate	3×10^{-4}
RIDE epochs	Number of gradient epochs per buffer update	5
RIDE batch size	Mini-batch size within each epoch	64

Bång B.8: CARE hyperparameters. The Beta Network alone determines the intrinsic-reward scale (no separate global coefficient α is used); the same Beta Network bounds and learning-progress hyperparameters apply to every CARE variant.

Symbol	**Description**	**Value**
$\beta_{\min}$	Lower bound on Beta Network output	0.001
$\beta_{\max}$	Upper bound on Beta Network output	0.1
β_0	Reference value for log-space regularization	0.01
γ_p	Progress-loss weight in L_β	1.0
λ_{reg}	Regularization weight in L_β	1×10^{-3}
ψ_{lr}	Beta Network learning rate	5×10^{-4}
Weight decay (ψ)	L_2 weight decay on Beta Network	1×10^{-6}
Gradient clipping (ψ)	Max L_2 norm on $\nabla_\psi L_\beta$	1.0
τ_{target}	Polyak soft-update rate for target Beta Network	0.01
ε	Numerical stability constant in standardization	1×10^{-8}

Bång B.9: Fixed-β sweep used in the baseline conditions $F_{m,\beta}$ of Section 4.4. In these conditions the Beta Network is replaced by the listed scalar value, the learning-progress update is disabled, and $\beta\, I_t^{+,m}$ is fed directly into the shaped reward.

Item	**Specification**
Modules swept	Count-based, ICM, RIDE
Sweep values β	$\{0.001, 0.005, 0.05, 0.1\}$
Conditions per module	4 fixed-β + 1 CARE = 5
Total intrinsic-augmented conditions	$3 \times 5 = 15$
Plus extrinsic-only control B_0	+1 (no intrinsic reward)
Total conditions per environment	16

Bång B.10: Other module-specific hyperparameters for count-based exploration and RIDE.

Symbol	**Description**	**Value**
d_{hash} (count)	Hash dimension for count-based exploration	32
Bonus type (count)	Form of count-based bonus	$1/\sqrt{N(s)}$
d_{hash} (RIDE)	Hash dimension for episodic count in RIDE	32
Δ (RIDE)	Bin width for episodic-count hash	0.1
RIDE encoder size	Latent dimension d_y of the RIDE encoder	256
RIDE encoder layers	Number of Linear $\rightarrow$ Tanh blocks in the encoder	2

Bảng B.11: Training schedule.

Symbol	**Description**	**Value**
max_ep_len	Maximum steps per episode	1,000
T	Rollout buffer length (PPO update period)	4,000
max_steps	Total environment interaction steps per seed	1,000,000
U	Total number of PPO updates per seed	250
seeds	Independent training seeds per configuration	$\{1, 2, 3, 4, 5\}$
Checkpoint interval	Frequency of model and trajectory saving	every 100,000 steps

The values listed in Tables B.1–B.11 are the canonical settings adopted across all benchmark MiniGrid environments. Environment-specific overrides, when present, are reported in Chapter 4.